

UnionLabs: new agent, features, physical layers

First Author

Institution
City, Country
first@example.edu

Second Author

Institution
City, Country
second@example.edu

Third Author

Institution
City, Country
third@example.edu

ABSTRACT

Running a wireless learning experiment still means hand-assembling everything around the algorithm: a radio stack, the wiring of every node, a testbed's containers, and the ports between them. We present UnionLabs, a platform that turns each of these into a designed, reusable part. An algorithm declares only *what to transmit* and *what to receive*, and moves unchanged between different physical layers. The wiring of a multi-node experiment is a declarative file in which every link carries its own medium, per direction—checked when it is read, not when it fails on the air. A shared workspace carries one account's experiment state across testbeds that cannot reach each other, and an agent turns any machine into a managed edge node, exposing hardware to experiment containers only by the lab owner's permission. In this demonstration, users create the experiment environment, visit the remote desktop in the container, and test the algorithms over different physical layers.

CCS CONCEPTS

• **Networks** → **Network experimentation.**

KEYWORDS

unionlabs, software-defined radio, wireless testbeds, experiment orchestration, wireless applications.

ACM Reference Format:

First Author, Second Author, and Third Author. 2026. UnionLabs: new agent, features, physical layers. In *Proceedings of The 32nd Annual International Conference on Mobile Computing and Networking (MobiCom '26)*. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. *MobiCom '26, October 26–30, 2026, Austin, TX, USA*
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 MOTIVATION

An experiment that places learning inside the radio loop, federated averaging over the air [6], decentralized consensus over a graph [5], is easy to state and expensive to run. Shared platforms have removed the hardware barrier [2, 3], and open physical-layer stacks the waveform barrier [1, 4], but users still need to understand the hardware code and link their algorithms to the physical layer, writing hundreds of lines and spending days debugging. We introduce UnionLabs, a platform that separates *what an algorithm does* from *how the experiment is wired*.

2 UNIONLABS DESIGN

The new features of UnionLabs comprise four parts, each shown in its own figure.

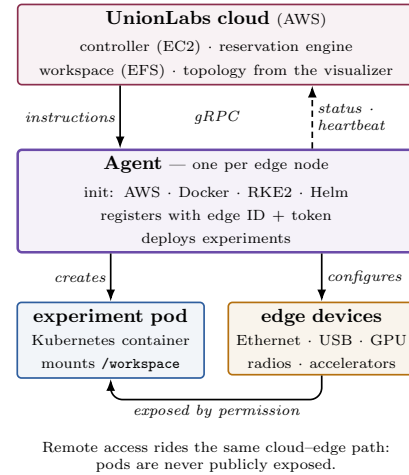


Figure 1: The UnionLabs Agent: middleware between the cloud services and the physical testbed.

The UnionLabs Agent. The agent runs on every edge node—a server, workstation, or laptop (Figure 1). At install it prepares AWS, Docker, an RKE2 Kubernetes server, and Helm, and registers the node with a unique edge ID and token. It then holds a gRPC channel with the cloud, where the controller, the reservation engine, and the topology uploaded through the testbed visualizer live: instructions flow down, status and heartbeats flow up. For each scheduled experiment, the agent creates a Kubernetes pod that mounts the shared workspace, configures the node's devices—Ethernet, USB radios, GPUs—and exposes them into the pod only by the

lab owner’s permission. Remote access rides the same cloud-edge path, so pods are never publicly exposed.

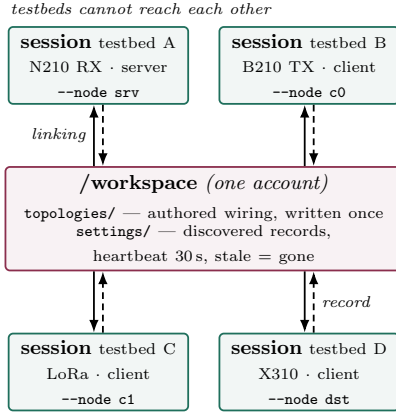


Figure 2: The shared workspace across sessions under a single account.

Deployment on Shared Testbeds. Every session of one account mounts the same /workspace, even when their testbeds cannot reach each other (Figure 2). Its topologies/ folder holds the authored wiring, written once by the experimenter; its settings/ folder holds the record each session publishes about itself, refreshed on a 30-second heartbeat and dropped when stale. The wiring links every session—here an N210 server, a B210 client, a LoRa client, and an X310 client on four testbeds—and their records flow back into the same folders.

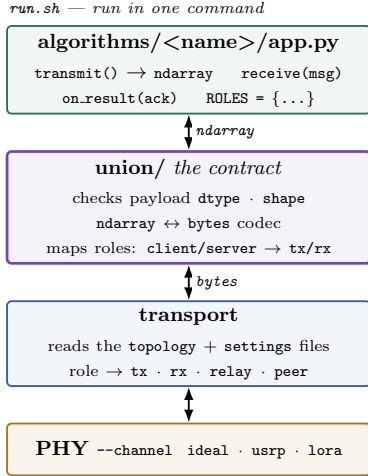


Figure 3: The union API. A uniform path to link the users’ algorithm with our embedded wireless system.

Algorithm Contract. The user’s algorithm writes two functions, transmit() and receive(msg), and nothing else (Figure 3). Beneath it, the union layer validates each payload and converts it between ndarray and bytes; the transport reads the topology and settings files and gives the node its

role—tx, rx, relay, or peer; and --channel selects the physical layer that carries the bytes. The same path runs in both directions, so an algorithm moves between physical layers by changing one flag, with no code change.

Wiring as a File. A topology is a declarative document naming each node, its role in the algorithm’s own vocabulary, the radio it owns, the ports it listens on, and the links between nodes; each node is launched with the same file and its own identity, and a typed flag still wins. One command checks the whole of it, walking 44 command-line flags and 82 topology settings from the text a user types to the object each configures.

Per-Link, Per-Direction Media. Each link states how it is carried, and its two directions may differ: a rig with one transmit-capable and one receive-only radio has exactly one RF path, so the reply must return over TCP/IP, and a chain may change medium half way through. Wirings that cannot run are refused when read—a transmitter with no transmit chain, two nodes claiming one port, or a reply asked to travel over the air.

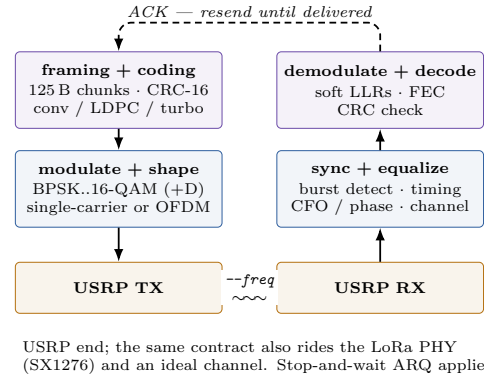


Figure 4: The PHY design, stop-and-wait ARQ applied.

PHY Codes. The transmit side frames each payload, protects it with a selectable channel code, and modulates it with a selectable scheme over a single-carrier or OFDM waveform (Figure 4). The receive side synchronizes to the burst, corrects carrier and phase, equalizes the channel, and decodes down to an integrity check. Stop-and-wait ARQ closes the loop: a chunk is resent until its acknowledgement arrives, over TCP or a second RF path. Modulation, coding, synchronization, waveform, gains, and the carrier frequency are all selectable per experiment, and the same contract also rides the LoRa PHY and an ideal, radio-free channel.

3 DEMONSTRATION

The demonstration shows each designed part working, live: visitors onboard a machine, open the experiment container, and run one algorithm over different physical layers. Each step takes two to three minutes.

Onboarding an Edge Node (Agent). A machine is enrolled on the spot: the agent prepares its environment, registers it, and its heartbeat appears on the platform. Once an experiment is scheduled, the visitor opens the container’s remote desktop through the cloud–edge path.

One Workspace Across Testbeds. From that desktop, the visitor edits a wiring file in /workspace and sees the same file from a session on a remote testbed, together with the records of the nodes that are live.

One Algorithm, Different Physical Layers. A learning algorithm runs first with no radio, then over real radios by changing one flag. Visitors watch delivery, retransmissions, and the airtime each round costs.

Rewiring, and Refusal. A multi-hop experiment changes its medium—over the air or over the network—as the visitor edits one field. A wiring that cannot run is refused when it is read, naming the node at fault.

REFERENCES

- [1] 2026. GNU Radio: The Free and Open Source Radio Ecosystem. <https://www.gnuradio.org>. Accessed: Month DD, 2026.
- [2] Leonardo Bonati, Salvatore D’Oro, Michele Polese, Stefano Basagni, and Tommaso Melodia. 2021. Colosseum: Large-Scale Wireless Experimentation Through Hardware-in-the-Loop Network Emulation. In *Proc. of IEEE DySPAN*.
- [3] Joe Breen, Andrew Buffmire, Jonathon Duerig, Kevin Dutt, Eric Eide, Mike Hibler, David Johnson, Sneha Kumar Kasera, Earl Lewis, Dustin Maas, et al. 2020. POWDER: Platform for Open Wireless Data-driven Experimental Research. In *Proc. of the 14th International Workshop on Wireless Network Testbeds, Experimental Evaluation & Characterization (WiNTECH)*.
- [4] Ismael Gomez-Miguel, Andres Garcia-Saavedra, Paul D. Sutton, Pablo Serrano, Cristina Cano, and Doug J. Leith. 2016. srsLTE: An Open-Source Platform for LTE Evolution and Experimentation. In *Proc. of the 10th ACM International Workshop on Wireless Network Testbeds, Experimental Evaluation & Characterization (WiNTECH)*.
- [5] Xiangru Lian, Ce Zhang, Huan Zhang, Cho-Jui Hsieh, Wei Zhang, and Ji Liu. 2017. Can Decentralized Algorithms Outperform Centralized Algorithms? A Case Study for Decentralized Parallel Stochastic Gradient Descent. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [6] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proc. of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 1273–1282.